

Vectorized LLMs: An Iterative Prompt-Inversion Framework for Emergent Model Synthesis

Steven Garner
Independent Research
Document ID: @13742x

August 2026

Abstract

Large language models (LLMs) are conventionally viewed as conditional generative systems that map an input prompt to an output distribution. Given a prompt p , a model M produces an output y ,

$$y \sim P_M(y \mid p). \quad (1)$$

This paper proposes a different architecture: **LLM vectorization through recursive prompt inversion**. Instead of treating the generated output as the endpoint of inference, the output is passed to a second model whose task is to infer the prompt that could have produced it. That inferred prompt is passed to a third model, which independently attempts to reconstruct the original prompt. The process continues through a chain of models until the inferred prompts converge.

The central hypothesis is that repeated convergence between forward generation and inverse prompt reconstruction provides a measurable representation of the semantic and functional behavior of a model. Multiple models can therefore be transformed into comparable behavioral vectors and subsequently synthesized into a new model.

Formally, the proposed architecture constructs an iterative system

$$p \xrightarrow{M_1} y_1 \xrightarrow{M_2^{-1}} \hat{p}_2 \xrightarrow{M_3} y_3 \xrightarrow{M_1^{-1}} \hat{p}_1 \longrightarrow \dots \quad (2)$$

until

$$d(p, \hat{p}_i) < \epsilon, \quad (3)$$

where d is a semantic distance function.

Once convergence has been established across a sufficiently broad prompt distribution, the participating models are represented as compatible behavioral operators. A new synthesized model M_Σ is then trained from these aligned representations. The synthesized model is subjected to the same recursive inversion process, allowing successive generations of models to be produced:

$$\mathcal{M}^{(0)} \rightarrow \mathcal{M}^{(1)} \rightarrow \mathcal{M}^{(2)} \rightarrow \dots \rightarrow \mathcal{M}^{(n)}. \quad (4)$$

The limiting objective is not merely to produce a larger model, but to construct an iterative process in which increasingly capable models emerge from the vectorized behavioral information of previous generations. This revised edition expands the original framework with a discussion of related work, a strengthened treatment of limitations, and a consolidated bibliography.

Contents

1 Introduction

2	Problem Definition	4
3	Forward and Inverse Operators	5
4	The Three-Model Convergence Architecture	5
5	Prompt Vectorization	6
6	Behavioral Equivalence	6
7	Convergence as a Fixed-Point Problem	6
8	Semantic Rather Than Literal Convergence	7
9	Reciprocal Model Testing	7
10	From Multiple Models to a Composite Model	8
11	Distillation Objective	8
12	Inverse Consistency Loss	8
13	Recursive Model Synthesis	9
14	Recursive Verification	9
15	Capability-Weighted Synthesis	9
16	The Model Population as a Vector Space	10
17	Interaction as an Information Source	10
18	Complementarity	10
19	Information-Theoretic Interpretation	11
20	Recursive Compression and Expansion	11
21	Emergent Model Generation	11
22	The Recursive Improvement Equation	12
23	An Alternative: Open-Ended Improvement	12
24	Preventing Degenerate Convergence	12
25	Adversarial Verification	12
26	Model Synthesis as an Optimization Problem	13
27	A Formal Definition of a Vectorized Model	13
28	Model Manifold	13

29 Recursive Vectorization Algorithm	14
30 Convergence Pseudocode	14
31 Why Three Models?	15
32 Consensus Matrix	15
33 From Consensus to Synthesis	16
34 Recursive Hierarchical Synthesis	16
35 The “Reversal” Principle	16
36 Connection to Autoencoding	17
37 Model Evolution	17
38 Theoretical Limit	17
39 Important Limitation: Agreement Is Not Capability	17
40 Experimental Hypotheses	18
41 Experimental Design	18
42 Evaluation Function	18
43 Scaling Law Hypothesis	19
44 Computational Complexity	19
45 Information-Gain Selection	19
46 Proposed Architecture	20
47 Proposed Name: Recursive LLM Vectorization	20
48 Proposed Name: Prompt-Reconstruction Consensus	20
49 Proposed End-State	21
50 Related Work	21
51 Conclusion	22
A Core Mathematical Definitions	23
B Compact Representation	23
C Suggested Research Program	23

1 Introduction

Modern LLM inference can be approximated as a forward transformation:

$$M : p \mapsto y. \quad (5)$$

A human provides a prompt p , and the model generates an output y . In probabilistic form,

$$P(y \mid p, M). \quad (6)$$

The conventional architecture therefore has a single principal direction:

$$p \rightarrow M \rightarrow y. \quad (7)$$

The proposed architecture introduces a second direction:

$$y \rightarrow M^{-1} \rightarrow p. \quad (8)$$

The inverse operation is not intended to reconstruct the exact sequence of characters originally supplied by the user. In general this is impossible because the forward mapping

$$f_M : p \rightarrow y \quad (9)$$

is many-to-one. Instead, the inverse model estimates a member of the equivalence class

$$[p]_M = \{p' : d(f_M(p'), f_M(p)) < \epsilon\}. \quad (10)$$

Thus, the relevant question becomes:

Can another model infer a prompt whose semantic and functional consequences are equivalent to those of the original prompt?

If the answer is repeatedly yes, the models demonstrate behavioral compatibility.

2 Problem Definition

Let the universe of prompts be $\mathcal{P}$ and the universe of possible outputs be $\mathcal{Y}$. An LLM M_i is modeled as a conditional probability operator

$$M_i : \mathcal{P} \rightarrow \Delta(\mathcal{Y}), \quad (11)$$

where $\Delta(\mathcal{Y})$ denotes the probability simplex over outputs. For a prompt p ,

$$y_i \sim M_i(\cdot \mid p). \quad (12)$$

A second model M_j performs prompt inversion:

$$\hat{p}_j \sim I_j(\cdot \mid y_i), \quad I_j : \mathcal{Y} \rightarrow \Delta(\mathcal{P}). \quad (13)$$

The reconstructed prompt is therefore

$$\hat{p}_j = \arg \max_{p' \in \mathcal{P}} P_{I_j}(p' \mid y_i). \quad (14)$$

The proposed system does not require $\hat{p}_j = p$. Instead, it requires semantic equivalence:

$$p \sim_\epsilon \hat{p}_j. \quad (15)$$

Define the equivalence relation

$$p \sim_\epsilon q \iff D(M(\cdot \mid p), M(\cdot \mid q)) < \epsilon. \quad (16)$$

The model is therefore judged by whether it preserves the **behavioral intent** of a prompt rather than its literal textual representation.

3 Forward and Inverse Operators

Define the forward operator $F_i(p) = M_i(p)$ and inverse operator $G_i(y) = I_i(y)$. A complete forward-inverse cycle is

$$p \xrightarrow{F_i} y \xrightarrow{G_j} \hat{p}. \quad (17)$$

The reconstruction error is

$$E_{ij}(p) = d_P(p, G_j(F_i(p))), \quad (18)$$

where d_P is a semantic distance over prompts. A successful inversion satisfies $E_{ij}(p) \rightarrow 0$.

However, literal prompt reconstruction is unnecessarily restrictive. We therefore define a semantic encoder $\phi_P : \mathcal{P} \rightarrow \mathbb{R}^d$ and an output encoder $\phi_Y : \mathcal{Y} \rightarrow \mathbb{R}^k$. Prompt distance becomes

$$d_P(p, q) = \|\phi_P(p) - \phi_P(q)\|_2. \quad (19)$$

Likewise,

$$d_Y(y, z) = \|\phi_Y(y) - \phi_Y(z)\|_2. \quad (20)$$

The inverse objective can then be written

$$G_j^* = \arg \min_G \mathbb{E}_{p \sim \mathcal{D}} [d_P(p, G(F_i(p)))]. \quad (21)$$

4 The Three-Model Convergence Architecture

The fundamental architecture uses three models. Let M_1, M_2, M_3 be three LLMs. An initial prompt is supplied: $p_0 = p$. Model M_1 generates

$$y_1 = F_1(p_0). \quad (22)$$

Model M_2 receives y_1 and attempts to determine the prompt that produced it:

$$p_1 = G_2(y_1). \quad (23)$$

Model M_3 independently receives the reconstructed prompt/output representation and attempts to determine the corresponding prompt:

$$p_2 = G_3(F_2(p_1)). \quad (24)$$

The cycle can be generalized as

$$p_{t+1} = G_{i_t}(F_{j_t}(p_t)). \quad (25)$$

Thus,

$$p_0 \rightarrow y_1 \rightarrow p_1 \rightarrow y_2 \rightarrow p_2 \rightarrow y_3 \rightarrow \dots. \quad (26)$$

The process terminates when $d_P(p_t, p_{t+1}) < \epsilon$ for K consecutive iterations. Define the convergence criterion

$$C(p) = \mathbf{1} \left[\max_{t \in T} d_P(p_t, p_{t+1}) < \epsilon \right]. \quad (27)$$

For a prompt distribution $\mathcal{D}$,

$$C_{\mathcal{D}} = \mathbb{E}_{p \sim \mathcal{D}} [C(p)]. \quad (28)$$

A high value of $C_{\mathcal{D}}$ indicates that the participating models can repeatedly translate between prompt space and output space without losing the underlying task specification.

5 Prompt Vectorization

The central concept of this paper is **prompt vectorization**. Rather than representing a prompt solely as a sequence of tokens, $p = (x_1, x_2, \dots, x_n)$, we represent its behavioral properties as a vector $v(p, M) \in \mathbb{R}^d$. Define

$$v_i(p) = \phi\left(F_i(p), G_i(F_i(p)), F_i(G_i(F_i(p)))\right). \quad (29)$$

The vector may contain multiple dimensions representing properties such as

$$v_i = [v_{\text{syntax}}, v_{\text{semantics}}, v_{\text{reasoning}}, v_{\text{factual}}, v_{\text{style}}, v_{\text{planning}}, v_{\text{coding}}, v_{\text{instruction}}, v_{\text{uncertainty}}, \dots]. \quad (30)$$

More generally,

$$v_i(p) = \begin{bmatrix} s_i(p) \\ r_i(p) \\ f_i(p) \\ c_i(p) \\ g_i(p) \\ u_i(p) \end{bmatrix}. \quad (31)$$

Here s_i represents semantic behavior, r_i reasoning behavior, f_i factual behavior, c_i compositional behavior, g_i generation characteristics, and u_i uncertainty characteristics. The entire model can then be approximated by a behavioral field

$$V_i = \{v_i(p) : p \in \mathcal{P}\}. \quad (32)$$

This is the **model vectorization**.

6 Behavioral Equivalence

Two models need not have identical parameters to behave similarly. Let M_a, M_b be two models. Define their behavioral distance as

$$D_B(M_a, M_b) = \mathbb{E}_{p \sim \mathcal{D}} [d_Y(F_a(p), F_b(p))]. \quad (33)$$

A stronger measure incorporates inverse reconstruction:

$$D_{BI}(M_a, M_b) = \mathbb{E}_{p \sim \mathcal{D}} [\alpha d_Y(y_a, y_b) + \beta d_P(p, \hat{p}_a) + \gamma d_P(p, \hat{p}_b)], \quad (34)$$

where $y_a = F_a(p)$, $y_b = F_b(p)$. The coefficients satisfy $\alpha + \beta + \gamma = 1$. Two models are behaviorally equivalent under tolerance ϵ when

$$D_{BI}(M_a, M_b) < \epsilon. \quad (35)$$

This gives a practical definition of model similarity independent of architecture, parameter count, tokenizer, or training history.

7 Convergence as a Fixed-Point Problem

The iterative system can be viewed as a dynamical system. Let $T = G \circ F$. Repeated reconstruction becomes

$$p_{t+1} = T(p_t). \quad (36)$$

A stable prompt representation is a fixed point $p^* = T(p^*)$, therefore $G(F(p^*)) = p^*$. For local convergence, consider the Jacobian

$$J_T(p) = \frac{\partial T(p)}{\partial p}. \quad (37)$$

A sufficient local condition for contraction is $\rho(J_T(p^*)) < 1$, where ρ denotes the spectral radius. If $\|J_T(p^*)\| < 1$, then small perturbations around p^* decrease under repeated application of T . Thus,

$$\lim_{t \rightarrow \infty} p_t = p^*. \quad (38)$$

This fixed-point formulation provides a mathematical foundation for the proposed convergence process, and places the framework in the same family as Banach-type fixed-point iterations used elsewhere in numerical analysis and in cycle-consistency formulations for cross-domain translation [15].

8 Semantic Rather Than Literal Convergence

A fundamental property of language is that many prompts can encode the same task. For example,

$$p_1 = \text{“Explain quantum mechanics simply.”} \quad (39)$$

and

$$p_2 = \text{“Give a beginner-friendly explanation of quantum mechanics.”} \quad (40)$$

may be different strings while being approximately equivalent in task space. Consequently, $p_1 \neq p_2$ but $\phi_P(p_1) \approx \phi_P(p_2)$. We define semantic convergence as

$$p_t \xrightarrow{\text{sem}} p^* \quad \text{when} \quad \|\phi_P(p_t) - \phi_P(p^*)\|_2 < \epsilon. \quad (41)$$

This distinction is essential because the proposed architecture seeks convergence of **information**, not necessarily convergence of textual strings.

9 Reciprocal Model Testing

The three-model system can be expressed as a closed loop:

$$M_1 \rightarrow M_2^{-1} \rightarrow M_3 \rightarrow M_1^{-1}. \quad (42)$$

For a prompt p ,

$$y_1 = M_1(p), \quad (43)$$

$$p_2 = M_2^{-1}(y_1), \quad (44)$$

$$y_3 = M_3(p_2), \quad (45)$$

$$p_1 = M_1^{-1}(y_3). \quad (46)$$

The loop error becomes

$$E_{\text{loop}} = d_P(p, p_1) + \lambda d_Y(y_1, y_3). \quad (47)$$

The normalized convergence score is

$$S = \exp(-E_{\text{loop}}), \quad 0 < S \leq 1. \quad (48)$$

A score near 1 indicates that the loop preserves the original task/output relationship.

10 From Multiple Models to a Composite Model

Suppose we have N models: $\mathcal{M} = \{M_1, M_2, \dots, M_N\}$. Each model is mapped to a behavioral vector $V_i = \Psi(M_i)$. We can construct a composite representation

$$V_\Sigma = \sum_{i=1}^N w_i V_i, \quad \text{subject to } w_i \geq 0, \sum_{i=1}^N w_i = 1. \quad (49)$$

The weights can depend on measured performance:

$$w_i = \frac{\exp(\tau Q_i)}{\sum_{j=1}^N \exp(\tau Q_j)}, \quad (50)$$

where Q_i is the measured capability of model M_i , and τ controls selection pressure (a softmax weighting scheme structurally related to temperature-scaled ensembling and to mixture-of-experts gating [10]). The composite behavioral field becomes

$$V_\Sigma(p) = \sum_{i=1}^N w_i V_i(p). \quad (51)$$

A new model M_Σ is then trained to approximate this composite field:

$$M_\Sigma \approx \Psi^{-1}(V_\Sigma). \quad (52)$$

This creates the first **synthetic generation**.

11 Distillation Objective

The new model should not merely reproduce one model’s outputs. Instead, it should minimize divergence from the ensemble distribution. Let $P_i(y | p)$ be the output distribution of model M_i . Define the ensemble distribution

$$P_\Sigma(y | p) = \sum_{i=1}^N w_i P_i(y | p). \quad (53)$$

The synthesized model is optimized by minimizing

$$\mathcal{L}_{\text{distill}} = \mathbb{E}_{p \sim \mathcal{D}} \left[D_{\text{KL}}(P_\Sigma(\cdot | p) \| P_{M_\Sigma}(\cdot | p)) \right], \quad (54)$$

following the general knowledge-distillation formulation introduced by Hinton et al. [2]. The objective can be augmented with behavioral reconstruction:

$$\mathcal{L} = \lambda_1 \mathcal{L}_{\text{distill}} + \lambda_2 \mathcal{L}_{\text{inverse}} + \lambda_3 \mathcal{L}_{\text{consistency}} + \lambda_4 \mathcal{L}_{\text{capability}}. \quad (55)$$

12 Inverse Consistency Loss

For every prompt p , generate $y = M_\Sigma(p)$. Then reconstruct $\hat{p} = I_\Sigma(y)$. Define

$$\mathcal{L}_{\text{inverse}} = d_P(p, \hat{p}). \quad (56)$$

A cycle-consistency loss is

$$\mathcal{L}_{\text{cycle}} = d_P(p, I_\Sigma(M_\Sigma(p))), \quad (57)$$

directly analogous to the cycle-consistency losses used in unpaired image-to-image translation [15]. The training objective therefore encourages

$$I_\Sigma \circ M_\Sigma \approx I_P, \quad (58)$$

where I_P is the identity operator over behavioral prompt representations.

13 Recursive Model Synthesis

After generating $M_\Sigma^{(1)}$, the same process is repeated. Define generation zero as the original collection: $\mathcal{M}^{(0)} = \{M_1, \dots, M_N\}$. The first synthesis produces

$$M^{(1)} = \text{SYNTH}(\mathcal{M}^{(0)}). \quad (59)$$

The second generation is

$$M^{(2)} = \text{SYNTH}(\mathcal{M}^{(1)} \cup \mathcal{M}^{(0)}). \quad (60)$$

More generally,

$$M^{(k+1)} = \text{SYNTH}(\mathcal{A}_k), \quad (61)$$

where $\mathcal{A}_k$ is the selected model population at generation k . The complete evolutionary process is

$$\boxed{\mathcal{M}^{(0)} \rightarrow M^{(1)} \rightarrow M^{(2)} \rightarrow M^{(3)} \rightarrow \dots} \quad (62)$$

and each newly synthesized model becomes subject to the same three-model verification procedure.

14 Recursive Verification

For generation k , define the verification operator $\mathcal{V}(M^{(k)})$. The model passes verification if $\mathcal{V}(M^{(k)}) \geq \theta$, where θ is the minimum convergence threshold. The recursive process is therefore

$$M^{(k)} \xrightarrow{\mathcal{V}} \begin{cases} \text{accepted,} & \mathcal{V}(M^{(k)}) \geq \theta, \\ \text{rejected / refined,} & \mathcal{V}(M^{(k)}) < \theta. \end{cases} \quad (63)$$

A generation can therefore be regarded as a hypothesis about the underlying behavioral function.

15 Capability-Weighted Synthesis

Simple averaging is insufficient because models possess different capabilities. Let

$$Q_i = \sum_{j=1}^K \alpha_j q_{ij} \quad (64)$$

represent the aggregate capability score of model i , where $\sum_{j=1}^K \alpha_j = 1$. Then

$$w_i = \frac{Q_i^\eta}{\sum_{j=1}^N Q_j^\eta}. \quad (65)$$

As $\eta \rightarrow \infty$, the system increasingly selects the highest-performing models. For finite η , knowledge from weaker models is retained. This produces an exploration/exploitation tradeoff:

$$\eta \downarrow \Rightarrow \text{greater diversity,} \quad \eta \uparrow \Rightarrow \text{greater selection pressure.} \quad (66)$$

16 The Model Population as a Vector Space

Let every model be represented by a vector $\mathbf{m}_i \in \mathbb{R}^d$. The model population becomes $\mathcal{M} = \{\mathbf{m}_1, \mathbf{m}_2, \dots, \mathbf{m}_N\}$. The synthesized model can be represented as

$$\mathbf{m}_\Sigma = \sum_{i=1}^N w_i \mathbf{m}_i + \delta, \quad (67)$$

where δ represents newly learned information — a decomposition conceptually related to task-vector arithmetic in weight space [4] and to Fisher-weighted model averaging [7]. This is important because the proposed system is not simply model averaging. If $\delta = 0$, the system is fundamentally interpolation. If $\delta \neq 0$, the system has an opportunity to learn novel representations from the interactions between models. The target is therefore

$$\boxed{\mathbf{m}_\Sigma = \text{Aggregate}(\mathbf{m}_1, \dots, \mathbf{m}_N) + \text{Learn}(\mathcal{I})} \quad (68)$$

where $\mathcal{I}$ represents information generated during model interaction.

17 Interaction as an Information Source

Suppose model M_i produces $y_i = F_i(p)$ and model M_j infers $\hat{p}_j = G_j(y_i)$. The pair generates an interaction record

$$r_{ij} = (p, y_i, \hat{p}_j). \quad (69)$$

For N models, the number of pairwise relationships is $R = \binom{N}{2} = \frac{N(N-1)}{2}$. If directed interactions are considered, $R_{\text{directed}} = N(N-1)$. Thus the interaction graph is $G = (V, E)$ where $V = \{M_1, \dots, M_N\}$. Each edge contains behavioral information:

$$e_{ij} = D(M_i, M_j). \quad (70)$$

The resulting matrix $\mathbf{D} = [D_{ij}]$ is a behavioral distance matrix. This matrix can be used to identify clusters, complementary models, redundant models, and outliers.

18 Complementarity

Two models need not be globally equivalent to be useful together. Define capability vector $\mathbf{q}_i = [q_{i1}, q_{i2}, \dots, q_{iK}]$. Model complementarity can be defined as

$$C_{ij} = 1 - \cos(\mathbf{q}_i, \mathbf{q}_j), \quad (71)$$

or alternatively $C_{ij} = \|\mathbf{q}_i - \mathbf{q}_j\|_2$. High complementarity means that two models contain different strengths. A synthesis objective can therefore maximize both capability and useful diversity:

$$J = \sum_i w_i Q_i + \lambda \sum_{i < j} w_i w_j C_{ij}. \quad (72)$$

The resulting population does not necessarily converge toward identical models; instead, it can preserve complementary capabilities while reducing behavioral inconsistency, echoing motivations behind multi-agent debate [1] and LLM ensembling/fusion approaches [5].

19 Information-Theoretic Interpretation

The system can also be described through mutual information [9]. Let $P \rightarrow Y \rightarrow \hat{P}$ be the forward/inverse chain. The objective is to maximize $I(P; \hat{P})$, the mutual information between the original prompt and reconstructed prompt. Recall

$$I(P; \hat{P}) = H(P) - H(P | \hat{P}). \quad (73)$$

Thus maximizing reconstruction quality corresponds to minimizing $H(P | \hat{P})$. However, literal reconstruction is not necessarily desirable. We therefore define a semantic variable Z :

$$P \rightarrow Z \rightarrow Y \rightarrow \hat{Z}. \quad (74)$$

The desired condition becomes

$$I(Z; \hat{Z}) \rightarrow H(Z) \iff H(Z | \hat{Z}) \rightarrow 0. \quad (75)$$

This states that the system should preserve the underlying task representation even if the exact wording changes.

20 Recursive Compression and Expansion

The proposed process can be interpreted as alternating compression and expansion. Forward generation performs $p \rightarrow y$; the output is a compressed representation of the task conditioned on the model’s learned knowledge. Inverse generation performs $y \rightarrow \hat{p}$. Repeated cycles produce

$$p_0 \rightarrow y_0 \rightarrow p_1 \rightarrow y_1 \rightarrow p_2 \rightarrow \dots. \quad (76)$$

If the system preserves task-relevant information, then $I(P; Y) \approx I(P; \hat{P})$. Information lost in the cycle is

$$L_I = I(P; Y) - I(P; \hat{P}). \quad (77)$$

The goal becomes $L_I \rightarrow 0$.

21 Emergent Model Generation

The central hypothesis of the framework is that sufficiently rich interaction records contain information not directly present in any single model representation. Let the original knowledge states be $K_1, K_2, \dots, K_N$. An interaction process generates

$$\mathcal{I} = \text{Interact}(K_1, \dots, K_N). \quad (78)$$

A synthesis model learns

$$K_{\text{new}} = f(K_1, \dots, K_N, \mathcal{I}). \quad (79)$$

The proposed system seeks

$$\mathcal{C}(K_{\text{new}}) > \max_i \mathcal{C}(K_i), \quad (80)$$

where $\mathcal{C}$ is a capability functional. This is the critical hypothesis of the architecture and must be experimentally validated rather than assumed.

22 The Recursive Improvement Equation

Define the capability of generation k as $Q_k = \mathcal{C}(M^{(k)})$. Recursive improvement occurs when $Q_{k+1} > Q_k$. Define improvement $\Delta_k = Q_{k+1} - Q_k$. The desired regime is $\Delta_k > 0$. An idealized system approaches

$$\lim_{k \rightarrow \infty} Q_k = Q^*. \quad (81)$$

If the capability ceiling is finite, $Q^* < \infty$. If each generation yields diminishing but positive improvement,

$$\Delta_k > 0, \quad \lim_{k \rightarrow \infty} \Delta_k = 0, \quad (82)$$

then $Q_k \rightarrow Q^*$. This represents convergence toward a capability frontier.

23 An Alternative: Open-Ended Improvement

The more ambitious hypothesis is $Q_{k+1} > Q_k \forall k$. This would imply $Q_0 < Q_1 < Q_2 < \dots$. If the capability metric is unbounded, $\lim_{k \rightarrow \infty} Q_k = \infty$. In practice, however, intelligence is multidimensional and no known scalar capability metric is guaranteed to be unbounded. Consequently, the scientifically testable objective should initially be:

$$\boxed{\mathbb{E}[Q_{k+1} - Q_k] > 0} \quad (83)$$

across independently measured capability domains.

24 Preventing Degenerate Convergence

A major risk is that the models agree simply because they make the same mistake. For example, $M_1(p) = y_{\text{wrong}}$, $M_2^{-1}(y_{\text{wrong}}) = p'$, and $M_3(p') = y_{\text{wrong}}$ could produce perfect internal agreement despite being externally incorrect. Therefore,

$$\text{agreement} \not\Rightarrow \text{truth}. \quad (84)$$

The system requires an external validation functional $T(y)$ and a composite score

$$Q_i = \alpha C_i + \beta A_i + \gamma R_i + \delta V_i, \quad (85)$$

where C_i is convergence, A_i is factual accuracy, R_i is reasoning quality, and V_i is external validation. A model must therefore satisfy $C_i \approx 1$ while simultaneously achieving $Q_i \geq \theta$.

25 Adversarial Verification

To prevent self-reinforcing errors, the system should introduce adversarial prompts. Let

$$\mathcal{D} = \mathcal{D}_{\text{normal}} \cup \mathcal{D}_{\text{adversarial}} \cup \mathcal{D}_{\text{novel}}. \quad (86)$$

The total objective becomes

$$\mathcal{L} = \sum_{d \in \mathcal{D}} \omega_d \mathcal{L}_d. \quad (87)$$

Adversarial prompts seek cases where $M_i \approx M_j$ but both are wrong. The system should therefore maximize Q_{true} rather than merely $Q_{\text{agreement}}$.

26 Model Synthesis as an Optimization Problem

The synthesis problem can be formulated as

$$M^* = \arg \min_M \mathbb{E}_{p \sim \mathcal{D}} [\mathcal{L}(M, p)]. \quad (88)$$

Let

$$\mathcal{L} = \lambda_1 L_{\text{output}} + \lambda_2 L_{\text{inverse}} + \lambda_3 L_{\text{agreement}} + \lambda_4 L_{\text{truth}} + \lambda_5 L_{\text{diversity}}. \quad (89)$$

Then

$$M^* = \arg \min_M [\lambda_1 L_o + \lambda_2 L_i + \lambda_3 L_a + \lambda_4 L_t + \lambda_5 L_d]. \quad (90)$$

This defines synthesis as a multi-objective optimization problem.

27 A Formal Definition of a Vectorized Model

We define the vectorization operator $\Psi : \mathcal{M} \rightarrow \mathbb{R}^d$ such that

$$\Psi(M) = \mathbb{E}_{p \sim \mathcal{D}} [\phi(p, M(p), I(M(p)))]. \quad (91)$$

The vectorized model is therefore

$$\mathbf{v}_M = \Psi(M) \quad (92)$$

and the distance between models becomes $d_M(M_i, M_j) = \|\Psi(M_i) - \Psi(M_j)\|_2$. More generally,

$$d_M(M_i, M_j) = \left((\mathbf{v}_i - \mathbf{v}_j)^T \Sigma^{-1} (\mathbf{v}_i - \mathbf{v}_j) \right)^{1/2}, \quad (93)$$

where Σ captures correlations between behavioral dimensions. This is a Mahalanobis behavioral distance.

28 Model Manifold

If models occupy a low-dimensional behavioral manifold $\mathcal{M} \subset \mathbb{R}^d$, then model synthesis becomes movement through that manifold. A model can be represented as $M(\mathbf{z})$ for latent behavioral coordinates $\mathbf{z} \in \mathbb{R}^r$, $r \ll d$. Synthesis then becomes

$$\mathbf{z}_{\text{new}} = f(\mathbf{z}_1, \dots, \mathbf{z}_N). \quad (94)$$

The objective is to locate

$$\mathbf{z}^* = \arg \max_{\mathbf{z}} \mathcal{C}(M(\mathbf{z})). \quad (95)$$

This suggests that sufficiently capable LLM populations might be analyzed geometrically rather than only by parameter count.

29 Recursive Vectorization Algorithm

The conceptual algorithm is given below.

Input: $\mathcal{M}^{(0)}, \mathcal{D}, \epsilon, \theta, K$

1. $p \sim \mathcal{D}$
2. $y_i = M_i(p)$
3. $\hat{p}_j = I_j(y_i)$
4. $\hat{y}_k = M_k(\hat{p}_j)$
5. repeat steps 3–4
6. until $d_P(p_t, p_{t-1}) < \epsilon$
7. calculate V_i
8. calculate Q_i
9. calculate w_i
10. $V_\Sigma = \sum_i w_i V_i$
11. $M^{(n+1)} = \text{SYNTH}(V_\Sigma)$
12. verify $M^{(n+1)}$
13. if $Q_{n+1} > Q_n$, continue.

30 Convergence Pseudocode

INPUT:

Models M1, M2, M3
 Prompt distribution D
 Semantic tolerance epsilon
 Validation threshold theta

FOR each prompt p in D:

current_prompt = p

REPEAT:

output_1 = M1(current_prompt)

 inferred_prompt_2 =
 InverseModel(M2, output_1)

 output_3 =
 M3(inferred_prompt_2)

 inferred_prompt_1 =
 InverseModel(M1, output_3)

```

    error =
        SemanticDistance(
            current_prompt,
            inferred_prompt_1
        )

    current_prompt =
        inferred_prompt_1

UNTIL error < epsilon

Store:
    prompt trajectory
    output trajectory
    convergence error
    external validation
    capability metrics

Vectorize models.
Calculate behavioral distances.
Calculate model weights.
Train synthesized model.
Repeat verification and synthesis.

```

31 Why Three Models?

A two-model system can establish $M_1 \rightarrow M_2^{-1}$, but this creates a potentially circular relationship. Three models introduce a third-party consistency condition:

$$M_1 \rightarrow M_2^{-1} \rightarrow M_3 \rightarrow M_1^{-1}. \quad (96)$$

The additional model acts as an independent witness. More generally, with N models, one can construct a graph $G = (V, E)$ and evaluate consistency across multiple paths. For a path $\pi = (M_{i_1}, M_{i_2}, \dots, M_{i_k})$, define

$$E_\pi = d_P(p, T_\pi(p)). \quad (97)$$

Global consistency becomes

$$E_G = \frac{1}{|\Pi|} \sum_{\pi \in \Pi} E_\pi. \quad (98)$$

This is stronger than testing a single chain.

32 Consensus Matrix

Define $A_{ij} = \exp(-D_{ij}/\sigma)$, so that $A_{ij} \in (0, 1]$ measures behavioral similarity. The matrix

$$A = \begin{bmatrix} 1 & A_{12} & \cdots & A_{1N} \\ A_{21} & 1 & \cdots & A_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ A_{N1} & A_{N2} & \cdots & 1 \end{bmatrix} \quad (99)$$

defines the model-consensus graph. Its normalized graph Laplacian is

$$L = I - D^{-1/2}AD^{-1/2}, \quad D_{ii} = \sum_j A_{ij}. \quad (100)$$

The eigenstructure of L can identify clusters of behaviorally similar models, in the same spirit as spectral clustering.

33 From Consensus to Synthesis

Suppose the model population separates into clusters $\mathcal{M} = \mathcal{C}_1 \cup \dots \cup \mathcal{C}_K$. Instead of blindly averaging all models, synthesize cluster representatives:

$$M_{\mathcal{C}_k} = \text{SYNTH}(\mathcal{C}_k). \quad (101)$$

Then synthesize the representatives:

$$M^* = \text{SYNTH}(M_{\mathcal{C}_1}, \dots, M_{\mathcal{C}_K}). \quad (102)$$

This hierarchical structure reduces redundancy and permits specialization.

34 Recursive Hierarchical Synthesis

The resulting architecture resembles a tree. At each level,

$$M^{(k+1)} = \text{SYNTH}(M_1^{(k)}, \dots, M_r^{(k)}). \quad (103)$$

Seven base models $M_1, \dots, M_7$ combine pairwise into intermediate nodes $M_{12}, M_{34}, M_{56}, M_7$, which are then merged into M_{new} . The depth of the hierarchy is

$$h = \lceil \log_r N \rceil. \quad (104)$$

35 The “Reversal” Principle

The proposed process can be described as a forward operation followed by reversal. Forward: $p \xrightarrow{M} y$. Reverse: $y \xrightarrow{I} p'$. If $p' \sim p$, then $I \circ M \approx I$. The synthesized model is subsequently required to reproduce the same property:

$$I_{\text{new}} \circ M_{\text{new}} \approx I. \quad (105)$$

Thus each generation inherits the requirement of reversibility. The stronger condition is $M \circ I \approx I_Y$, giving two-sided approximate invertibility:

$$\boxed{I \circ M \approx I_P, \quad M \circ I \approx I_Y.} \quad (106)$$

36 Connection to Autoencoding

The architecture can be interpreted as a language-model autoencoder:

$$p \xrightarrow{M} y \xrightarrow{I} \hat{p}. \quad (107)$$

The difference is that the encoder and decoder are themselves independent LLMs. A multi-model system becomes

$$p \xrightarrow{M_1} y_1 \xrightarrow{I_2} p_2 \xrightarrow{M_3} y_3 \xrightarrow{I_4} p_4. \quad (108)$$

The system therefore creates a distributed semantic autoencoder in which multiple independent models participate in reconstruction.

37 Model Evolution

Let $k = \text{Vectorize}(\mathcal{M}^{(k)})$. Synthesis becomes $M^{(k+1)} = \text{Learn}(k)$. The evolutionary operator is $: \mathcal{M}^{(k)} \rightarrow \mathcal{M}^{(k+1)}$. Recursive application gives

$$\mathcal{M}^{(n)} =^n (\mathcal{M}^{(0)}). \quad (109)$$

A stable model population satisfies $(\mathcal{M}^*) = \mathcal{M}^*$. An improving system satisfies

$$\mathcal{C}((\mathcal{M})) > \mathcal{C}(\mathcal{M}). \quad (110)$$

38 Theoretical Limit

If the synthesis operator is monotonic under capability measure $\mathcal{C}$,

$$\mathcal{C}((M)) \geq \mathcal{C}(M), \quad (111)$$

then $Q_{k+1} \geq Q_k$. If strict improvement occurs infinitely often, $Q_{k+1} > Q_k$ for infinitely many k . A practical stopping condition is

$$|Q_{k+1} - Q_k| < \delta \quad (112)$$

for r successive generations. This produces a convergence condition for the model-generation process.

39 Important Limitation: Agreement Is Not Capability

The central scientific danger is the assumption $\text{consensus} \Rightarrow \text{intelligence}$. This implication is not guaranteed. A group of models can share training-data biases, hallucinations, reasoning errors, blind spots, stylistic conventions, and incorrect assumptions. Therefore,

$$D(M_i, M_j) \rightarrow 0 \quad (113)$$

does not necessarily imply $\mathcal{C}(M_i) \rightarrow \max$. The proposed architecture must therefore distinguish three quantities:

Agreement $\neq$ Accuracy $\neq$ Capability.

(114)

The goal is to maximize all three simultaneously.

40 Experimental Hypotheses

The framework produces several experimentally testable hypotheses.

H1 — Prompt inversion. For sufficiently capable models, $\mathbb{E}[d_P(p, I(M(p)))]$ will decrease as model capability increases.

H2 — Cross-model agreement. For models with similar capabilities, $D_{BI}(M_i, M_j)$ will be lower than for randomly selected models.

H3 — Vectorization. Behavioral vectors $\Psi(M_i)$ will predict model performance better than simple parameter count alone.

H4 — Synthesis. A synthesized model trained on vectorized interaction data will outperform at least some of its constituent models.

H5 — Recursive improvement. Repeated synthesis will produce $Q_{k+1} > Q_k$ on independently held-out evaluations.

H6 — Diversity advantage. A heterogeneous model population will produce a better synthesized model than a homogeneous population of equivalent total compute.

41 Experimental Design

A first experiment can use $N = 3$ models. For each prompt $p_i \sim \mathcal{D}$, record

$$R_i = (p_i, y_i, \hat{p}_i, \hat{y}_i, s_i). \quad (115)$$

The dataset becomes $\mathcal{R} = \{R_1, \dots, R_n\}$. Measure

$$E_{\text{prompt}} = \frac{1}{n} \sum_i d_P(p_i, \hat{p}_i), \quad (116)$$

$$E_{\text{output}} = \frac{1}{n} \sum_i d_Y(y_i, \hat{y}_i), \quad (117)$$

$$Q_{\text{external}} = \frac{1}{n} \sum_i q(y_i). \quad (118)$$

The synthesized model is then evaluated on an unseen test distribution $\mathcal{D}_{\text{test}} \cap \mathcal{D}_{\text{train}} = \emptyset$.

42 Evaluation Function

Define

$$Q(M) = w_c C(M) + w_a A(M) + w_r R(M) + w_f F(M) + w_g G(M), \quad \sum_i w_i = 1, \quad (119)$$

where C is cycle consistency, A is accuracy, R is reasoning, F is factuality, and G is generalization. The key comparison is

$$Q(M_{\text{new}}) \stackrel{?}{>} \max_i Q(M_i). \quad (120)$$

43 Scaling Law Hypothesis

Suppose capability follows a scaling relation $Q(N) = aN^\alpha + b$, consistent with the empirically observed neural scaling laws for language models [3, 6]. The recursive synthesis hypothesis predicts an additional generation-dependent term:

$$Q(N, k) = aN^\alpha + b + \gamma k^\beta. \quad (121)$$

If $\gamma > 0$, recursive synthesis contributes capability beyond ordinary scale. A stronger hypothesis is

$$Q(N, k) = aN^\alpha + \gamma \log(1 + k) + b. \quad (122)$$

Empirical measurements can determine whether $\gamma > 0$ and whether the improvement saturates.

44 Computational Complexity

For N models and P prompts, pairwise interaction has complexity approximately $O(PN^2)$. If each interaction requires L generated tokens,

$$C_{\text{interaction}} = O(PN^2L). \quad (123)$$

Recursive generations produce

$$C_{\text{total}} = \sum_{k=0}^K O(P_k N_k^2 L_k). \quad (124)$$

This can become computationally expensive. Consequently, the architecture should eventually use selective interaction:

$$E^* = \arg \max_{E \subseteq V \times V} \text{InformationGain}(E). \quad (125)$$

The system should preferentially interact models whose outputs are both (1) high quality and (2) behaviorally complementary.

45 Information-Gain Selection

Define the information gain of interaction (i, j) :

$$IG_{ij} = H(\Theta) - H(\Theta \mid R_{ij}), \quad (126)$$

where Θ represents uncertainty about the target behavioral model. Select

$$(i, j)^* = \arg \max_{i, j} IG_{ij}. \quad (127)$$

This transforms the system from exhaustive model comparison into active learning, drawing on the same information-gain criteria used in Bayesian experimental design and active learning [8].

46 Proposed Architecture

The complete system can therefore be expressed as five layers:

$$\boxed{\text{Prompt Layer} \rightarrow \text{Generation Layer} \rightarrow \text{Inverse Layer} \rightarrow \text{Vector Layer} \rightarrow \text{Synthesis Layer}} \quad (128)$$

with an external validation loop:

$$\boxed{\text{Validation} \rightarrow \text{Selection} \rightarrow \text{Synthesis} \rightarrow \text{Validation.}} \quad (129)$$

The complete architecture is

$$\begin{aligned} p &\rightarrow \{M_i(p)\}_{i=1}^N \\ &\rightarrow \{\hat{p}_{ij}\} \\ &\rightarrow \{V_i\} \\ &\rightarrow \{D_{ij}\} \\ &\rightarrow \{w_i\} \\ &\rightarrow V_\Sigma \\ &\rightarrow M_{\text{new}} \\ &\rightarrow \mathcal{V}(M_{\text{new}}) \\ &\rightarrow M_{\text{new}}. \end{aligned} \quad (130)$$

47 Proposed Name: Recursive LLM Vectorization

The methodology can be named **Recursive LLM Vectorization (RLV)**. The operator is

$$\text{RLV} : M \mapsto V(M) \mapsto M'. \quad (131)$$

For a population, $\text{RLV}(\mathcal{M}) = M'$. Recursive application gives $M_{k+1} = \text{RLV}(M_k)$. The intended long-term objective is

$$\lim_{k \rightarrow \infty} \mathcal{C}(M_k) = \mathcal{C}^*, \quad (132)$$

where $\mathcal{C}^*$ is the maximum capability reachable by the synthesis process.

48 Proposed Name: Prompt-Reconstruction Consensus

The three-model verification process can separately be called **Prompt-Reconstruction Consensus (PRC)**. For models M_1, M_2, M_3 ,

$$\text{PRC}(M_1, M_2, M_3, p) = \text{Consensus}(p_1, p_2, p_3). \quad (133)$$

A consensus score is

$$S_{\text{PRC}} = 1 - \frac{\sum_{i < j} d_P(p_i, p_j)}{\binom{N}{2}}. \quad (134)$$

The system accepts a model population when $S_{\text{PRC}} > \theta$.

49 Proposed End-State

The ultimate system is not one model querying another indefinitely without control. Instead, it is a **recursive model-evolution architecture**:

$$\boxed{\text{Generate} \rightarrow \text{Invert} \rightarrow \text{Compare} \rightarrow \text{Vectorize} \rightarrow \text{Synthesize} \rightarrow \text{Validate} \rightarrow \text{Repeat}} \quad (135)$$

At generation k , $M^{(k)} \rightarrow V^{(k)} \rightarrow M^{(k+1)}$. The desired condition is

$$\mathcal{C}(M^{(k+1)}) > \mathcal{C}(M^{(k)}), \quad \text{while maintaining} \quad E_{\text{cycle}}(M^{(k+1)}) < \epsilon. \quad (136)$$

Thus, every new generation must simultaneously become **more capable and more behaviorally coherent**.

50 Related Work

The framework proposed here sits at the intersection of several established research threads, none of which alone captures the recursive prompt-inversion mechanism proposed above, but each of which motivates or constrains individual components of the architecture.

Knowledge distillation and ensembling. The distillation objective in Section 12 generalizes the teacher-student framework of Hinton et al. [2] to a weighted mixture of teachers. Ensemble-of-LLMs approaches such as LLM-Blender [5] and multi-agent debate [1] similarly combine multiple model outputs, but typically operate at inference time rather than producing a single synthesized parametric model.

Model merging and weight-space arithmetic. Sections 16 and 28 describe the synthesized model as a point in a behavioral vector space. This is conceptually related to model soups [14], task-arithmetic editing [4], and Fisher-weighted merging [7], all of which combine model *parameters* directly. The proposed framework differs in that it vectorizes *behavior* (via forward/inverse prompt cycles) rather than weights, which in principle allows synthesis across models with incompatible architectures or tokenizers.

Cycle consistency. The inverse-consistency loss in Section 13 and the reversal principle in Section 35 are directly modeled on cycle-consistency losses from unpaired image translation [15]. That literature provides existing theoretical treatment of when $I \circ M \approx \text{id}$ is achievable and when it degenerates (e.g., steganographic encoding of information invisible to the loss function), which is directly relevant to the degenerate-convergence risk discussed in Section 24.

Self-consistency and chain-of-thought. The convergence criterion in Section 4 resembles self-consistency decoding [12], in which multiple reasoning paths from a single model are sampled and reconciled by majority vote. The present framework extends this idea across models rather than across samples from one model, and substitutes a learned inverse operator for a simple voting rule.

Scaling laws. Section 43’s scaling-law hypothesis builds on the empirical power-law relationships between compute, data, parameter count, and loss established by Kaplan et al. [6] and refined by Hoffmann et al. [3]. The proposed generation-dependent term γk^β is a novel, untested extension specific to this framework and should be treated as a hypothesis rather than an established result.

Active learning and information gain. The selective-interaction criterion in Section 45 reuses the information-gain objective common to Bayesian active learning [8], applied here to the choice of which model pairs should be made to interact.

None of the above works propose the specific recursive forward/inverse convergence loop of Sections 4 and 9, nor the recursive re-synthesis of Section 37. The framework should therefore be read as a novel synthesis of existing techniques rather than a validated method; Section 40’s hypotheses (H1–H6) are offered as the falsifiable core of the proposal.

51 Conclusion

This paper proposes a framework in which LLMs are treated not merely as generators but as reversible behavioral systems. The conventional relationship $p \rightarrow M \rightarrow y$ is extended into

$$p \rightarrow M_1 \rightarrow y \rightarrow M_2^{-1} \rightarrow \hat{p} \rightarrow M_3 \rightarrow \hat{y} \rightarrow M_1^{-1} \rightarrow \hat{\hat{p}}. \quad (137)$$

Repeated application seeks a fixed point:

$$\lim_{t \rightarrow \infty} p_t = p^*. \quad (138)$$

The converged behavior is then encoded as a vector: $M_i \mapsto \mathbf{v}_i$. Multiple model vectors can be combined according to capability and complementarity:

$$\mathbf{v}_\Sigma = \sum_i w_i \mathbf{v}_i. \quad (139)$$

A new model is trained to reproduce the resulting behavioral field: $M_\Sigma = \Psi^{-1}(\mathbf{v}_\Sigma)$. The new model is then subjected to the same recursive verification:

$$M_\Sigma \rightarrow \text{PRC} \rightarrow \text{Vectorize} \rightarrow \text{Synthesize}. \quad (140)$$

The entire process forms a recursive operator:

$$M^{(k+1)} = \text{SYNTH} \left(\text{VECTORIZE} \left(\text{VERIFY}(M^{(k)}) \right) \right). \quad (141)$$

The long-term hypothesis is

$$\boxed{\mathcal{C}(M^{(k+1)}) > \mathcal{C}(M^{(k)})} \quad (142)$$

for successive generations.

However, the most important scientific qualification is that **agreement between models cannot itself establish intelligence, truth, or superiority**. The system therefore requires independent evaluation, adversarial testing, factual validation, and held-out benchmarks.

If those conditions are satisfied, Recursive LLM Vectorization provides a potentially interesting research direction: treating collections of language models as a population of behavioral operators, extracting their latent capabilities through forward/inverse interaction, and recursively synthesizing those capabilities into new generations of models.

The fundamental research question can ultimately be reduced to one experimentally testable inequality:

$$\boxed{\mathbb{E} \left[\mathcal{C}(\text{SYNTH}(M_1, \dots, M_N)) \right] > \max_i \mathcal{C}(M_i)} \quad (143)$$

If this inequality can be demonstrated repeatedly under independent evaluation, then recursive vectorization would constitute more than ensemble agreement or ordinary distillation: it would provide evidence for a mechanism through which **model populations can recursively generate increasingly capable model representations**.

A Core Mathematical Definitions

$$\text{Forward operator} \quad F_M(p) = y \quad (144)$$

$$\text{Inverse operator} \quad G_M(y) = \hat{p} \quad (145)$$

$$\text{Cycle} \quad T_M(p) = G_M(F_M(p)) \quad (146)$$

$$\text{Fixed point} \quad T_M(p^*) = p^* \quad (147)$$

$$\text{Cycle error} \quad E_M(p) = d_P(p, T_M(p)) \quad (148)$$

$$\text{Behavioral vector} \quad V_M = \Psi(M) \quad (149)$$

$$\text{Behavioral distance} \quad D(M_i, M_j) = \|\Psi(M_i) - \Psi(M_j)\| \quad (150)$$

$$\text{Ensemble vector} \quad V_\Sigma = \sum_i w_i V_i \quad (151)$$

$$\text{Ensemble output distribution} \quad P_\Sigma(y \mid p) = \sum_i w_i P_i(y \mid p) \quad (152)$$

$$\text{Synthesis} \quad M_\Sigma = \arg \min_M D(M, V_\Sigma) \quad (153)$$

$$\text{Recursive synthesis} \quad M_{k+1} = \text{SYNTH}(M_k) \quad (154)$$

$$\text{Capability improvement} \quad \Delta_k = \mathcal{C}(M_{k+1}) - \mathcal{C}(M_k) \quad (155)$$

$$\text{Target condition} \quad \Delta_k > 0 \quad (156)$$

B Compact Representation

The central recursive architecture can be summarized as

$$\boxed{\begin{array}{l} p \xrightarrow{M_1} y_1 \xrightarrow{I_2} p_2 \xrightarrow{M_3} y_3 \xrightarrow{I_4} p_4 \\ \xrightarrow{d_P < \epsilon} p^* \\ \xrightarrow{\Psi} V \\ \xrightarrow{\text{SYNTH}} M_{\text{new}} \\ \xrightarrow{\text{PRC}} \text{verified} \\ \xrightarrow{\text{RLV}} M_{\text{next}}. \end{array}} \quad (157)$$

The recursive process is

$$\boxed{M^{(0)} \xrightarrow{\text{RLV}} M^{(1)} \xrightarrow{\text{RLV}} M^{(2)} \xrightarrow{\text{RLV}} \dots} \quad (158)$$

with the desired property

$$\boxed{\mathcal{C}(M^{(0)}) < \mathcal{C}(M^{(1)}) < \mathcal{C}(M^{(2)}) < \dots} \quad (159)$$

subject to independent validation.

C Suggested Research Program

The concept should be tested in progressively more demanding stages:

$$\text{Prompt inversion} \rightarrow \text{Cross-model inversion} \rightarrow \text{Three-model consensus} \rightarrow \text{Behavioral vectorization} \rightarrow \text{Model synthesis} \quad (160)$$

The decisive experiment is not whether the models eventually agree. It is whether a synthesized generation achieves

$$Q_{\text{new}} > \max(Q_1, Q_2, \dots, Q_N) \quad (161)$$

on **previously unseen tasks**, including tasks specifically designed to expose shared errors. If that result is reproducible, the proposed framework becomes a substantive research program rather than simply another ensemble or model-distillation technique.

References

- [1] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch. Improving Factuality and Reasoning in Language Models through Multiagent Debate. *arXiv preprint arXiv:2305.14325*, 2023.
- [2] G. Hinton, O. Vinyals, and J. Dean. Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531*, 2015.
- [3] J. Hoffmann, S. Borgeaud, A. Mensch, et al. Training Compute-Optimal Large Language Models. *arXiv preprint arXiv:2203.15556*, 2022.
- [4] G. Ilharco, M. T. Ribeiro, M. Wortsman, S. Gururangan, L. Schmidt, H. Hajishirzi, and A. Farhadi. Editing Models with Task Arithmetic. *International Conference on Learning Representations (ICLR)*, 2023.
- [5] D. Jiang, X. Ren, and B. Y. Lin. LLM-Blender: Ensembling Large Language Models with Pairwise Ranking and Generative Fusion. *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.
- [6] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling Laws for Neural Language Models. *arXiv preprint arXiv:2001.08361*, 2020.
- [7] M. S. Matena and C. Raffel. Merging Models with Fisher-Weighted Averaging. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [8] B. Settles. Active Learning Literature Survey. *Computer Sciences Technical Report 1648, University of Wisconsin–Madison*, 2009.
- [9] C. E. Shannon. A Mathematical Theory of Communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
- [10] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. *International Conference on Learning Representations (ICLR)*, 2017.
- [11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention Is All You Need. *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [12] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-Consistency Improves Chain of Thought Reasoning in Language Models. *International Conference on Learning Representations (ICLR)*, 2023.
- [13] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [14] M. Wortsman, G. Ilharco, S. Y. Gadre, R. Roelofs, R. Gontijo-Lopes, A. S. Morcos, H. Namkoong, A. Farhadi, Y. Carmon, S. Kornblith, and L. Schmidt. Model Soups: Averaging Weights of Multiple Fine-Tuned Models Improves Accuracy without Increasing Inference Time. *International Conference on Machine Learning (ICML)*, 2022.

- [15] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros. Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks. *IEEE International Conference on Computer Vision (ICCV)*, 2017.

Document metadata. Author: Steven Garner. Title: *Vectorized LLMs: An Iterative Prompt-Inversion Framework for Emergent Model Synthesis*. Date: August 2026. Identifier: @13742x. This document presents a theoretical framework and a set of falsifiable hypotheses (Section 40); no experimental results are reported herein. Readers intending to implement the described architecture should treat Sections 40–41 as the minimum required validation protocol before drawing conclusions about recursive capability improvement.